KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"SYNTAX ERROR DETECTION AND RECOVERY SYSTEM"

Submitted as a part of the course

Compiler Design

Submitted By

Student Name	Enrollment No.
Vyom Patel	202307020084
Priti Sisodiya	20230702088
Krushil Rafaliya	202307020090

Program: B.Tech CSE-AIML
Semester: VII

Under the Guidance of

Ms Neha Minocha
Assistant Professor
Department of Computer Science & Engineering Technology

Karnavati University, Gandhinagar

TABLE OF CONTENTS

1. ABSTRACT 1

2. INTRODUCTION 2

2.1 Compiler and its Phases 2
2.2 Role of the Syntax Analyser 2
2.3 Types of Errors in a Program 3
2.4 Goals of an Error Handler 3
2.5 Problem Statement 3
2.6 Objectives 4
2.7 Scope of the Project 4
2.8 Organisation of the Report 4

3. OVERVIEW OF PROJECT 5

3.1 System Overview 5
3.2 System Architecture 5
3.3 The MiniC Language 6
3.4 Error Recovery Strategies 7
3.5 Recovery Modes 10
3.6 Corrected Program Generation 11
3.7 Table-Driven LL(1) Parser Demonstrator 11
3.8 User Interface 12

4. TECHNOLOGY USED 13

4.1 Programming Language: Python 3 13
4.2 GUI Toolkit: Tkinter 13
4.3 Python Standard Library Modules 13
4.4 Software and Hardware Requirements 14
4.5 Compiler Design Concepts Applied 14
4.6 Why a Hand-Written Parser instead of Lex/Yacc 14
4.7 Project Structure 15

5. FLOW CHART OF THE PROJECT 16

5.1 Overall Flow of the System 16
5.2 Phrase-Level Recovery in expect() 17
5.3 Panic Mode Recovery 17
5.4 LL(1) Parsing with Panic-Mode Recovery 18

6. SCREENSHOTS OF THE PROJECT 19

6.1 Diagnostics 19
6.2 Corrected Code 19
6.3 Statistics and Comparison of Modes 20
6.4 Parse Tree 21
6.5 Recovery Log 21

6.6 LL(1) Parser: Parsing Trace 22
6.7 LL(1) Parser: Parsing Table 23
6.8 No-Recovery Mode 23
6.9 Command-Line Interface 24
7. RESULTS AND CONCLUSION 25
7.1 Test Programs 25
7.2 Results in Full-Recovery Mode 25
7.3 Comparison of Recovery Modes 26
7.4 Example: Before and After Recovery 26
7.5 Testing 27
7.6 Performance 27
7.7 Limitations 28
7.8 Conclusion 28
7.9 Future Scope 28
REFERENCES 29

1. ABSTRACT

Every real program is written with mistakes, and a compiler is judged as much by how it handles wrong input as by how it translates correct input. A compiler that stops at the first syntax error forces the programmer into a slow edit–compile–fix loop, while one that recovers poorly floods the screen with misleading, cascading error messages. Error detection and recovery is therefore one of the most practical responsibilities of the syntax analysis phase of a compiler.

This project presents the **Syntax Error Detection and Recovery System (SEDRS)**, a compiler front end for *MiniC*, a small C-like language designed for the project. SEDRS performs lexical analysis, a global delimiter-balance analysis and recursive-descent syntax analysis. When it finds an error it reports the exact line and column, a clear message and a suggested fix, and then **recovers** so that the rest of the program is still analysed. It implements all four classic recovery strategies described in compiler textbooks — **panic mode**, **phrase-level recovery**, **error productions** and a restricted form of **global correction** — together with lexical error recovery.

Every repair made during recovery is recorded, and at the end SEDRS produces an **automatically corrected program**, which is re-parsed to confirm that it contains zero errors. The system can run in three modes (full recovery, panic mode only and no recovery) so that the benefit of each strategy can be compared on the same input. A separate **table-driven LL(1) parser** module computes FIRST and FOLLOW sets, builds the predictive parsing table with *synch* entries and shows a step-by-step trace of panic-mode recovery, exactly as taught in the course.

The system is written in Python 3 using only the standard library, with a Tkinter graphical interface and a command-line interface for demonstration. It was evaluated on eight sample programs covering every class of error; in all cases it detected every error, recovered and generated a corrected program that compiles cleanly. All 40 automated unit tests pass, and analysis of each sample takes about one millisecond.

Keywords: compiler design, syntax analysis, error detection, error recovery, panic mode, phrase-level recovery, error productions, global correction, recursive-descent parser, LL(1) parsing, FIRST and FOLLOW sets.

2. INTRODUCTION

2.1 Compiler and its Phases

A compiler is a program that reads a program written in a source language and translates it into an equivalent program in a target language. An important part of this job is to report to the user any errors found in the source program. A compiler is organised as a sequence of phases, each of which transforms the program from one representation to another. The first three phases — lexical, syntax and semantic analysis — form the **front end** (analysis part), and the remaining phases form the **back end** (synthesis part).

Table 2.1: Phases of a compiler

Phase	Input	Output	Main task
Lexical analysis	Character stream	Token stream	Groups characters into tokens (keywords, identifiers, numbers, operators)
Syntax analysis	Token stream	Parse tree	Checks that tokens follow the grammar of the language
Semantic analysis	Parse tree	Annotated tree	Type checking, declaration-before-use, scope rules
Intermediate code generation	Annotated tree	Three-address code	Machine-independent representation
Code optimisation	Intermediate code	Optimised code	Makes the code faster or smaller
Code generation	Optimised code	Target code	Produces machine or assembly code

SEDRS implements the first two phases of this pipeline — lexical analysis and syntax analysis — and concentrates on what these phases must do when the input is **not** correct.

2.2 Role of the Syntax Analyser

The syntax analyser (parser) obtains a string of tokens from the lexical analyser and verifies that the string can be generated by the grammar of the source language. For a well-formed program it builds a parse tree and passes it to the rest of the compiler. For an ill-formed program it must report the syntax errors in an intelligible way and **recover** from commonly occurring errors so that it can continue processing the remainder of the program. Because most programming-language errors are syntactic in nature and because parsing methods such as LL and LR detect an error as soon as possible (the *viable-prefix property*), the parser is the natural place for error handling.

2.3 Types of Errors in a Program

Programming errors can occur at different levels. The table below summarises the four common classes and the phase that detects each of them.

Table 2.2: Classes of programming errors

Error class	Example in MiniC	Detected by
Lexical	int x = 5 @ 3; (illegal character), "hello (unterminated string), 3.14.15	Lexical analyser
Syntactic	missing ;, unbalanced ( ), if x > 3 {, misspelled keyword whlie	Syntax analyser
Semantic	type mismatch, using an undeclared variable	Semantic analyser
Logical	< used instead of <=, infinite loop	Not detected by the compiler

SEDRS deals with the first two classes — lexical and syntactic errors — which together form the large majority of errors a programmer meets when compiling a program.

2.4 Goals of an Error Handler

According to Aho, Lam, Sethi and Ullman, the error handler in a parser has goals that are simple to state but challenging to realise:

1. **Report** the presence of errors clearly and accurately — the right line and column, and a message that explains the problem.
2. **Recover** from each error quickly enough to detect subsequent errors — the parser should not stop at the first error, and it should not produce a flood of spurious messages caused by the first one.
3. **Add minimal overhead** to the processing of correct programs.

SEDRS is designed around these three goals, and the results in Chapter 7 measure how well each one is met.

2.5 Problem Statement

Design and implement a syntax error detection and recovery system for a small C-like language that (a) detects lexical and syntax errors with precise positions and helpful messages, (b) recovers from them using the standard recovery strategies so that all errors in a program are reported in a single run, (c) avoids cascading errors and always terminates, (d) produces a corrected version of the program, and (e) provides an interface through which the working of each strategy can be demonstrated.

2.6 Objectives

- Design the MiniC language and its grammar, and build a lexer and a recursive-descent parser for it.

- Detect lexical and syntax errors with exact line/column positions and hints for fixing them.
- Implement panic-mode, phrase-level, error-production and global (delimiter) recovery, plus lexical error recovery.
- Suppress cascading (spurious) errors and guarantee that parsing always terminates.
- Generate a corrected program from the recorded repairs and verify that it re-parses with zero errors.
- Compare full recovery, panic mode only and no recovery on the same program.
- Demonstrate the textbook table-driven LL(1) panic-mode algorithm using FIRST/FOLLOW *synch* sets.
- Provide a GUI and a CLI through which the practical working of the project can be demonstrated.

2.7 Scope of the Project

In scope: the MiniC language (data types int, float, char, bool, void; one-dimensional arrays; functions; if-else, while, for, print, return, break, continue; C-style expressions with the usual precedence), lexical and syntactic error handling, corrected-code generation, and an LL(1) demonstrator for user-supplied grammars.

Out of scope: semantic analysis (type checking and symbol-table based checks), intermediate code generation, optimisation and code generation. These later phases are not needed to demonstrate syntax error detection and recovery, and are listed as future work in Chapter 7.

2.8 Organisation of the Report

Chapter 3 gives an overview of the project: the architecture, the MiniC language and each recovery strategy. Chapter 4 describes the technology used. Chapter 5 presents the flowcharts of the system and of its main algorithms. Chapter 6 contains screenshots of the working system. Chapter 7 presents the results, testing, conclusion and future scope.

3. OVERVIEW OF PROJECT

3.1 System Overview

SEDRS takes a MiniC source program as input — opened from a file, chosen from the built-in samples or typed directly into the editor — and produces the following outputs:

- a **list of diagnostics**: for each error its severity (error, warning or note), phase, line and column, message, the recovery strategy used, the action taken and a suggestion;
- the **token table** produced by the lexer;
- the **parse tree** built by the parser (error nodes are marked);
- the **corrected program**, with every repair highlighted;
- a **recovery log** showing the step-by-step actions of the parser;
- **statistics**: errors per strategy and a comparison of the three recovery modes.

The key idea of the system is that every phase *never stops* on bad input. Each phase records what went wrong and how it was repaired, and then continues. This lets one run of the compiler report all the errors in a program, which is exactly what a programmer needs.

3.2 System Architecture

SEDRS is organised as a pipeline of independent modules. Every phase appends to a shared list of diagnostics and of repairs (text edits); the repair engine finally applies the edits to produce the corrected program. Table 3.1 lists the modules and their responsibilities.

Table 3.1: Modules of SEDRS

Module (file)	Responsibility
sedrs/tokens.py	Token class, keyword and operator sets, synchronizing token set
sedrs/lexer.py	Scanner using maximal munch; detects and repairs lexical errors
sedrs/delimiters.py	Stack-based ( ) [ ] { } matching over the whole file; reports where an unmatched bracket was opened
sedrs/parser.py	Recursive-descent parser for MiniC; implements phrase-level, error-production and panic-mode recovery; builds the parse tree
sedrs/repair.py	Applies repairs (insert / delete / replace / comment-out) to the source
sedrs/diagnostics.py	Diagnostic and Repair records, names of the recovery strategies
sedrs/ll1.py	Generic FIRST/FOLLOW computation, LL(1) table with synch entries, conflict detection, panic-mode trace
sedrs/analyzer.py	Facade that runs all phases, re-parses the corrected program and collects statistics
gui.py	Tkinter graphical user interface
main.py	Command-line interface and program entry point

Module (file)	Responsibility
tests/test_sedrs.py	40 automated unit tests

3.3 The MiniC Language

MiniC is a small subset of C chosen so that it contains every construct where typical syntax errors occur — declarations, assignments, function calls, conditions, loops, blocks and expressions — while remaining small enough to explain completely.

3.3.1 Tokens

Table 3.2: Token classes of MiniC

Token class	Members / pattern
Type keywords	int, float, char, bool, void
Statement keywords	if, else, while, for, return, print, break, continue
Literal keywords	true, false
Identifier	letter or _ followed by letters, digits or _
Number	integer (42) or floating point (3.14)
String / char literal	"text" / 'c'
Operators	== != <= >= && || ++ -- += -= + - * / % = < > !
Delimiters	; , ( ) { } [ ]
Comments	// line comment and / *block comment* /

The lexer uses the **maximal munch** rule: the operator list is ordered longest-first, so <= is read as one token rather than < followed by =.

3.3.2 Grammar

The grammar used by the recursive-descent parser is given below in EBNF. { } means zero or more repetitions and [ ] means optional.

```
program     -> { function | statement } EOF
function    -> type ID '(' [ params ] ')' block
params      -> 'void' | type ID { ',' type ID }
statement   -> declaration ';' | assignment ';' | call ';'
             | if_stmt | while_stmt | for_stmt | block
             | 'print' '(' [ args ] ')' ';' | 'return' [ expr ] ';'
             | 'break' ';' | 'continue' ';' | ';'
declaration -> type ID [ '[' NUM ']' ] [ '=' expr ] { ',' ID [ '=' expr ] }
assignment  -> ID [ '[' expr ']' ] ( '=' | '+=' | '-=' ) expr
             | ID ( '++' | '--' )
if_stmt     -> 'if' '(' expr ')' statement [ 'else' statement ]
while_stmt  -> 'while' '(' expr ')' statement
for_stmt    -> 'for' '(' [ declaration | assignment ] ';' [ expr ] ';'
               [ assignment ] ')' statement
block       -> '{' { statement } '}'
expr        -> or        or  -> and { '||' and }     and -> eq { '&&' eq }
eq          -> rel { ('==' | '!=') rel }
rel         -> add { ('<' | '>' | '<=' | '>=') add }
add         -> mul { ('+' | '-') mul }
mul         -> unary { ('*' | '/' | '%') unary }
unary       -> ('!' | '-') unary | primary
primary     -> NUM | FLOAT | STRING | CHAR | 'true' | 'false'
             | ID [ '(' [ args ] ')' | '[' expr ']' ] | '(' expr ')'
```

Each non-terminal is implemented by one method of the Parser class (for example statement(), declaration(), if_statement(), expression()). Operator precedence is handled by one level of the expression grammar per precedence level, from || (lowest) to unary operators (highest).

3.4 Error Recovery Strategies

Once an error is detected the parser must bring itself into a state from which parsing can continue. Compiler textbooks describe four standard strategies. SEDRS implements all four, plus recovery inside the lexer. Table 3.3 compares them.

Table 3.3: Error recovery strategies

Strategy	Idea	Strength / weakness
Panic mode	Discard input symbols until a synchronizing token (such as ; or }) is found, then resume.	Simple and can never loop; but skips input and may miss errors in it.
Phrase level	Make a local correction to the remaining input: insert, delete or replace one token.	Precise and fast; must be used carefully to avoid infinite loops.

Strategy	Idea	Strength / weakness
Error productions	Augment the grammar with productions that generate common erroneous constructs.	Very good messages for expected mistakes; only covers what the designer foresaw.
Global correction	Choose a minimal sequence of changes that makes the whole input correct.	Theoretically best; too expensive in general. Used here in a restricted form (delimiter balancing).

3.4.1 Lexical Error Recovery

The lexer never stops on bad input. For each lexical error it records a diagnostic and a repair, makes the smallest sensible change and continues scanning. Table 3.4 shows the lexical errors handled.

Table 3.4: Lexical errors and recovery actions

Erroneous input	Message	Recovery action
@ # $	illegal character	character deleted
a & b, a | b	single '&' / '|'	replaced by && / ||
"hello	unterminated string literal	closing quote inserted at end of line
/* comment ...	unterminated block comment	*/ inserted at end of file
3.14.15	malformed number	extra part deleted (3.14)
2ndValue	identifier starts with a digit	accepted as identifier _2ndValue
'ab'	malformed character literal	truncated to 'a'

3.4.2 Phrase-Level Recovery

Every terminal the parser requires is consumed through a single routine, expect(kind). When the current token does not match, expect tries three local corrections in a fixed order before giving up:

- **Substitution** — a token that is a common confusion for the expected one is replaced, for example , instead of ; in for (i = 0, i < n, i++), or] instead of).
- **Deletion** — if the token *after* the current one is the expected token, the current token is extraneous and is deleted, e.g. the second) in print(x));.
- **Insertion** — a missing ;,),], (or } is inserted virtually, but only when the context makes it safe. A ; is inserted only if the next token starts a new line, is a keyword, a } or the end of the file.

The safe-insertion rule is what keeps phrase-level recovery from looping or guessing wrongly. For int d = a b; it refuses to insert a ; between a and b and instead reports *missing operator*. Missing ; errors are reported at the end of the previous token (for example line 7, column 18),

which is where the programmer expects them, rather than at the first token of the next line. The same phrase-level idea is used when a statement ends before its body: for while (x) at the end of the file or just before a }, an empty block { } is inserted. A simplified version of expect is shown below.

```
def expect(self, kind, context):
    t = self.cur
    if t.kind == kind:                      # normal case
        return self.advance()
    if self.full:
        if (kind, t.kind) in SUBSTITUTIONS:    # (a) substitution
            ...replace t with kind...
        if self.peek().kind == kind:           # (b) deletion
            ...delete t...
        if self.can_insert(kind):              # (c) insertion
            ...insert virtual token after previous token...
    self.error(f"expected {kind} {context} ...")   # -> panic mode
```

3.4.3 Error Productions

The grammar is augmented with rules that recognise frequent mistakes and give specific messages. These rules are implemented directly inside the parsing methods. Table 3.5 lists them.

Table 3.5: Error productions implemented

Erroneous input	Message produced	Recovery action
whlie (x) / pritn(x) / itn a / retrun 0 / fi (c)	unknown identifier 'whlie' - did you mean keyword 'while'?	treat the identifier as the keyword
if x > 3 {	'if' condition must be enclosed in parentheses	insert (and)
while y < 10) {	missing '(' before 'while' condition	insert (
if (x = 5)	assignment '=' used inside a condition	interpret as ==
x == 10;	comparison '==' used as a statement	interpret as =
else without if	'else' without a matching 'if'	ignore else, parse its body
a + * b	unexpected operator '*' after '+'	delete the extra operator
print("x" y)	missing ',' between arguments	insert ,
if (c);	warning: empty body after 'if' (stray ';')	accept ; as an empty statement

Misspelled keywords are detected with the *optimal string alignment* edit distance (Levenshtein distance extended with transposition of two adjacent characters). An identifier is treated as a keyword only if the distance is at most 1 for keywords of up to four letters and at most 2 for longer ones, and only if the keyword fits the context. Two checks prevent false alarms on ordinary identifiers: a name that the program declares as a function is never flagged, and a call-shaped statement such as foo(); (a name, a bracketed argument list and then a ;) can only be a misspelling of print, never of if, while or for, because those keywords must be followed by a body rather than a ;.

3.4.4 Panic Mode Recovery

If no local correction applies, expect() raises a ParseError. It is caught by the statement() wrapper, which skips tokens until it reaches the **synchronizing set**:

```
SYNC_TOKENS = { ';', '}', '{', EOF } | TYPE_KEYWORDS | (STMT_KEYWORDS -
{'else'})
        = { ;  }  {  EOF  int float char bool void
           if while for return print break continue }
```

A ; is consumed because it ends the broken statement; the other synchronizing tokens are left in place for the enclosing rule. The discarded statement is turned into a comment in the corrected program, and any local repairs made inside it are dropped. Two safeguards make panic mode robust:

- **Guaranteed termination** — every loop that parses a list of statements forces at least one token of progress, so the parser can never loop forever.
- **Cascade suppression** — a second panic error at the same token position as the previous one is treated as a cascading error and suppressed instead of being reported again.

3.4.5 Global Delimiter Analysis

A left-to-right parser discovers a missing) or } late, often far away from the real cause. Before parsing, SEDRS makes one stack-based pass over all tokens that matches brackets and reports the **root cause** as a note, for example *"'(' opened at line 9 is not closed before '{' at line 9"* or *"'{' opened at line 2 is never closed"*. Because (and [can never span a block, they are reported at the first { or } that follows them. This is a restricted but practical form of global correction: it looks at the whole program at once instead of only at the current token.

3.5 Recovery Modes

The same program can be analysed in three modes, which makes it easy to show why recovery matters.

Table 3.6: Recovery modes

Mode	Strategies active	Behaviour
Full recovery	Lexical, phrase level, error productions, panic mode, global delimiter analysis	Finds all errors, repairs locally where possible, produces a corrected program
Panic mode only	Lexical recovery and panic mode	Finds most errors but discards whole statements; may produce spurious errors
No recovery	None	Behaves like a naive compiler: reports the first syntax error and stops

3.6 Corrected Program Generation

Every recovery action records a **repair** — an edit to the original source text with a position, a length and replacement text. Four kinds of repair are used: *insert* (for example a missing ;), *delete* (an extra)), *substitute* (whlie to while) and *discard* (a statement thrown away by panic mode is wrapped in /* ... */). After parsing, apply_repairs() sorts the repairs by position and applies them, skipping edits that overlap a larger one that was already applied. The result is the corrected program, together with the positions of the changed text so that the GUI can highlight them in green. Finally the analyser **re-parses the corrected program** to verify that it contains zero errors.

3.7 Table-Driven LL(1) Parser Demonstrator

To demonstrate the textbook algorithm, ll1.py accepts any grammar typed by the user. It computes FIRST and FOLLOW sets by fixed-point iteration, builds the predictive parsing table M[A, a], detects LL(1) conflicts, and fills every empty entry M[A, b] with **synch** for each terminal b in FOLLOW(A). The default grammar is the classic expression grammar:

```
E  -> T E'
E' -> + T E' | - T E' | ε
T  -> F T'
T' -> * F T' | / F T' | ε
F  -> ( E ) | id | num
```

Table 3.7: FIRST and FOLLOW sets of the expression grammar

Non-terminal	FIRST	FOLLOW
E	{ (, id, num }	{), $ }
E'	{ +, -, ε }	{), $ }
T	{ (, id, num }	{ +, -,), $ }

Non-terminal	FIRST	FOLLOW
T'	{ *, /, ε }	{ +, -,), $ }
F	{ (, id, num }	{ +, -, *, /,), $ }

During parsing, the following panic-mode rules are applied:

- If the top of the stack is a terminal that does not match the input, it is popped (as if it had been inserted).
- If M[A, a] is empty, the input symbol a is skipped.
- If M[A, a] is synch, the non-terminal A is popped.

For the input a + * b) * 3, the parser detects four errors — the * after +, and the extra), * and 3 after a complete expression — recovers from each of them and accepts the input. The complete trace is shown in Chapter 6.

3.8 User Interface

The project has two interfaces through which its working can be demonstrated.

Graphical interface (Tkinter). The window has a source editor with line numbers on the left and a set of tabs on the right. Erroneous lines are highlighted in red with a dot in the gutter, and hovering over red code shows the error message. The tabs are: *Diagnostics*, *Tokens*, *Parse Tree*, *Corrected Code*, *Recovery Log*, *Statistics* and *LL(1) Parser* (with sub-tabs FIRST & FOLLOW, Parsing table M[A, a] and Parsing trace). The toolbar has Open, Save, a sample selector, the recovery-mode switch (Full / Panic only / None), a *Live* option that re-analyses the program while typing, *Analyze (F5)* and *Apply Fixes*, which loads the corrected program back into the editor.

Table 3.8: Command-line options

Command	Purpose
python main.py	launch the GUI
python main.py FILE	analyse FILE and print a coloured report
--mode full | panic | none	choose the recovery mode
--tokens --tree --log	also print the token table, parse tree and recovery log
--compare	run all three modes and compare the results
--fix OUTFILE	write the corrected program to OUTFILE
--ll1 "EXPR"	run the table-driven LL(1) parser on an expression
python -m unittest discover -s tests -v	run the 40 unit tests

4. TECHNOLOGY USED

4.1 Programming Language: Python 3

SEDRS is implemented in **Python 3** (version 3.8 or newer). Python was chosen for the following reasons:

- **Readability** — the code of the lexer and the parser reads almost like the grammar itself, which makes the recovery logic easy to explain and evaluate.
- **Fast development** — data classes, sets and dictionaries make it simple to represent tokens, FIRST/FOLLOW sets and the parsing table.
- **Portability** — the same program runs on Windows, macOS and Linux without compilation.
- **Batteries included** — the GUI toolkit and the unit-testing framework are part of the standard library, so no third-party package has to be installed.

4.2 GUI Toolkit: Tkinter

The graphical interface is built with **Tkinter** and its themed widget set **ttk**, which ship with Python. The editor is a tk.Text widget with tags for syntax colouring, error highlighting and repair highlighting; the diagnostics, token and LL(1) tables use ttk.Treeview; the parse tree is a collapsible tree view; the line-number gutter is drawn on a tk.Canvas; and the statistics chart is drawn with simple coloured bars.

4.3 Python Standard Library Modules

Table 4.1: Standard library modules used

Module	Used for
tkinter, ttk, filedialog, messagebox	Graphical user interface
dataclasses	Token, Diagnostic, Repair and AnalysisResult records
argparse	Command-line options in main.py
unittest	Automated test suite (40 tests)
collections.Counter	Counting errors per recovery strategy
itertools	Sequence numbers that keep repairs in a stable order
time	Measuring analysis time (perf_counter)
re	Syntax colouring in the editor
os, sys	File paths, console encoding and colours
ctypes	Sharp rendering on high-DPI screens (Windows only)

4.4 Software and Hardware Requirements

Table 4.2: Software requirements

Item	Requirement
Operating system	Windows 10/11 (run.bat provided), macOS or Linux
Language runtime	Python 3.8 or newer
Libraries	Python standard library only (Tkinter included); no third-party packages
Launch	run.bat (Windows) or python main.py / python3 main.py

Table 4.3: Hardware requirements

Item	Requirement
Processor	Any processor that can run Python 3.8
Memory	4 GB RAM is more than sufficient (analysis takes about 1 ms per program)
Display	1280 × 720 or higher recommended for the GUI

4.5 Compiler Design Concepts Applied

Table 4.4: Course concepts and where they are used

Concept	Where it is used in SEDRS
Tokens, lexemes, maximal munch	lexer.py — scanning MiniC source into tokens
Context-free grammar (EBNF)	MiniC grammar implemented by parser.py
Recursive-descent (top-down) parsing	One method per non-terminal in parser.py
Operator precedence and associativity	Layered expression grammar (or, and, eq, rel, add, mul, unary)
Parse tree	Node objects built by the parser and shown in the GUI
Panic mode, phrase level, error productions, global correction	parser.py and delimiters.py
Stack-based bracket matching	delimiters.py
FIRST and FOLLOW sets	ll1.py — fixed-point computation
Predictive (LL(1)) parsing table, synch entries	ll1.py — table construction and panic-mode parsing
LL(1) conflict detection	ll1.py — reports grammars that are not LL(1)

5. FLOW CHART OF THE PROJECT

5.1 Overall Flow of the System

Figure 5.1 shows the complete flow of SEDRS from input to output. The source program first passes through lexical analysis, where lexical errors are repaired on the spot. The global delimiter analysis then reports the root cause of any unbalanced brackets. The recursive-descent parser processes the program statement by statement; on each syntax error it either halts (no-recovery mode) or recovers and continues. Finally the repairs are applied to generate the corrected program and all results are displayed.

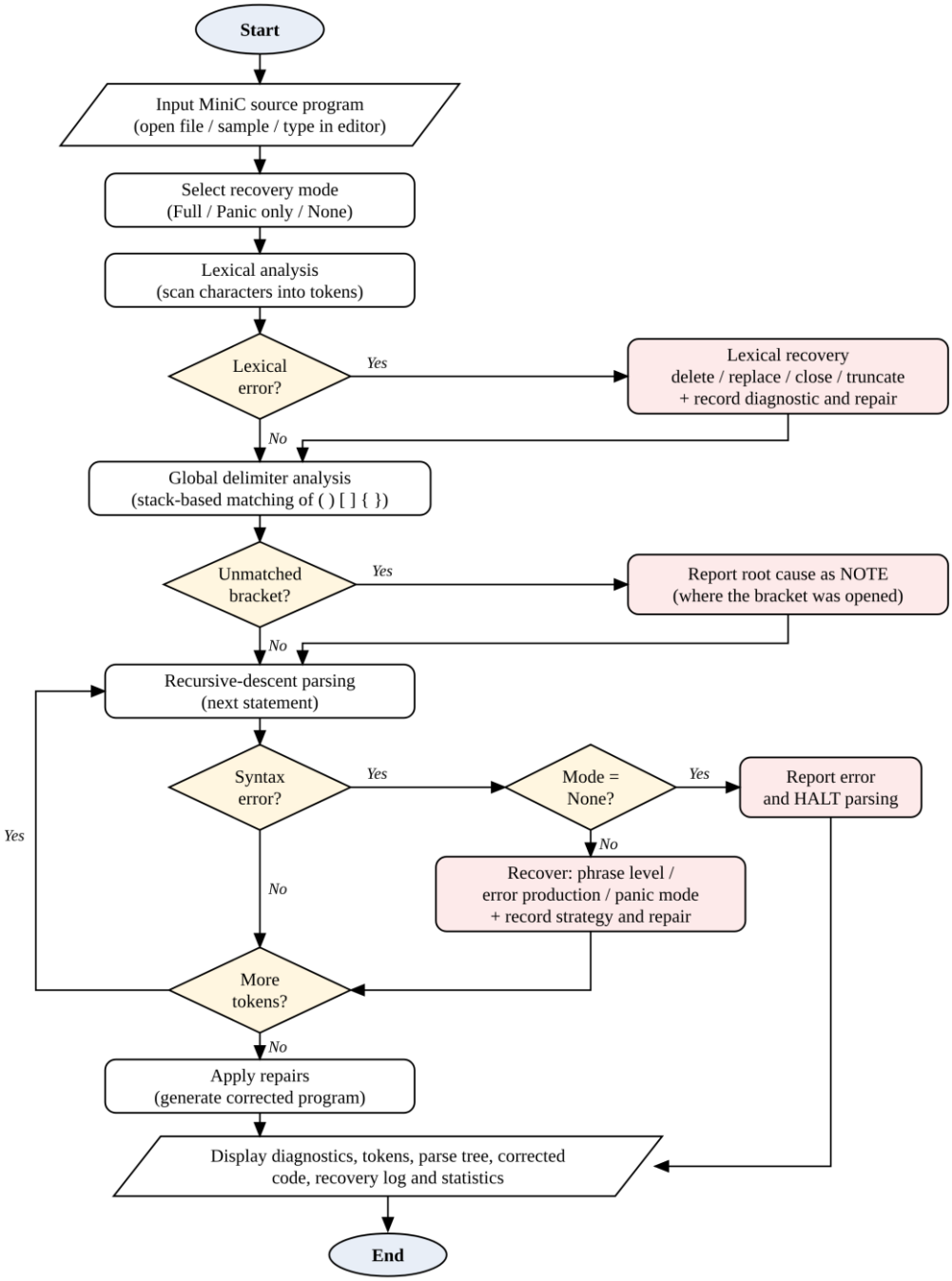


Figure 5.1: Overall flowchart of SEDRS

5.2 Phrase-Level Recovery in expect()

Figure 5.2 shows the decision sequence inside expect(). The three local corrections are tried in a fixed order — substitution, deletion, insertion — and only if none of them is safe does the parser fall back to panic mode.

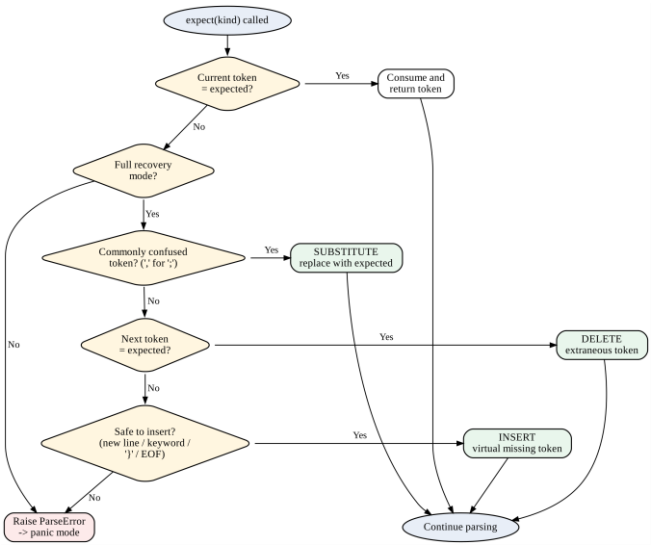


Figure 5.2: Flowchart of phrase-level recovery in expect()

5.3 Panic Mode Recovery

Figure 5.3 shows how a ParseError is handled. Cascading errors are suppressed, progress is forced so that parsing always terminates, and tokens are skipped until a synchronizing token is reached. The discarded text is commented out in the corrected program.

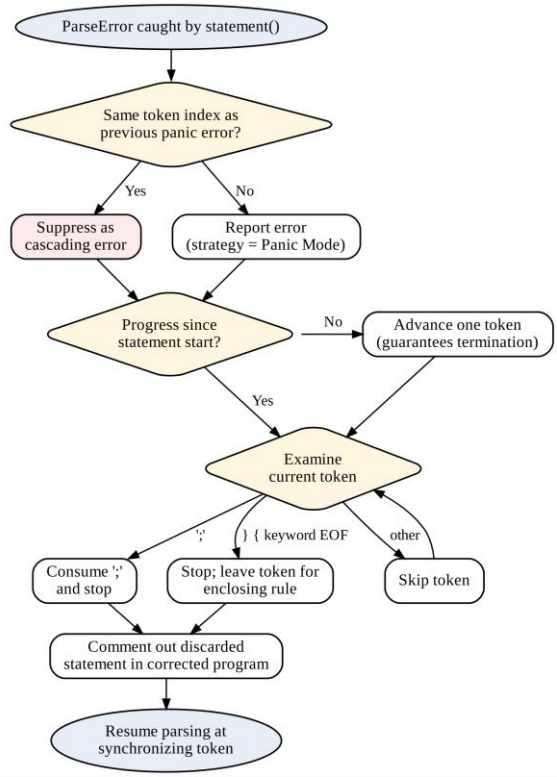


Figure 5.3: Flowchart of panic mode recovery

5.4 LL(1) Parsing with Panic-Mode Recovery

Figure 5.4 shows the table-driven LL(1) algorithm with the three panic-mode error rules: pop an unmatched terminal, pop a non-terminal on a synch entry and skip the input symbol on an empty entry.

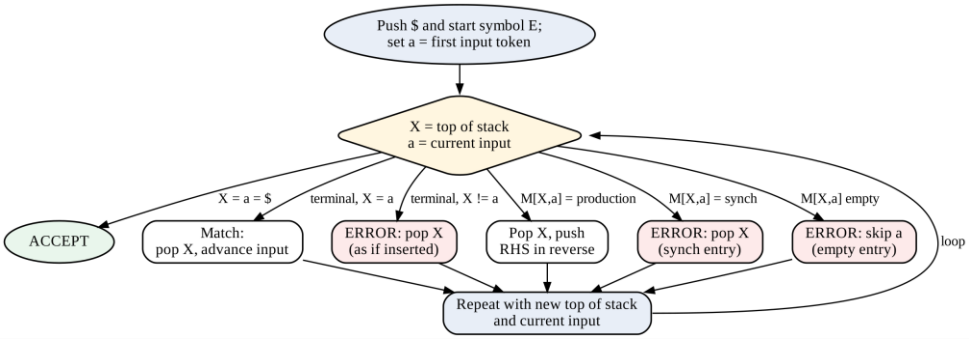


Figure 5.4: Flowchart of LL(1) parsing with panic-mode recovery

6. SCREENSHOTS OF THE PROJECT

This chapter shows the working of the project. Unless stated otherwise the screenshots use sample 08_mixed_errors.mc, which contains ten different errors in one program.

6.1 Diagnostics

The main window. Erroneous lines are highlighted in red with a dot in the gutter. The Diagnostics tab lists every error with its line and column, phase, message and recovery strategy; the panel below gives the details of the selected error, including the action taken and a suggestion.

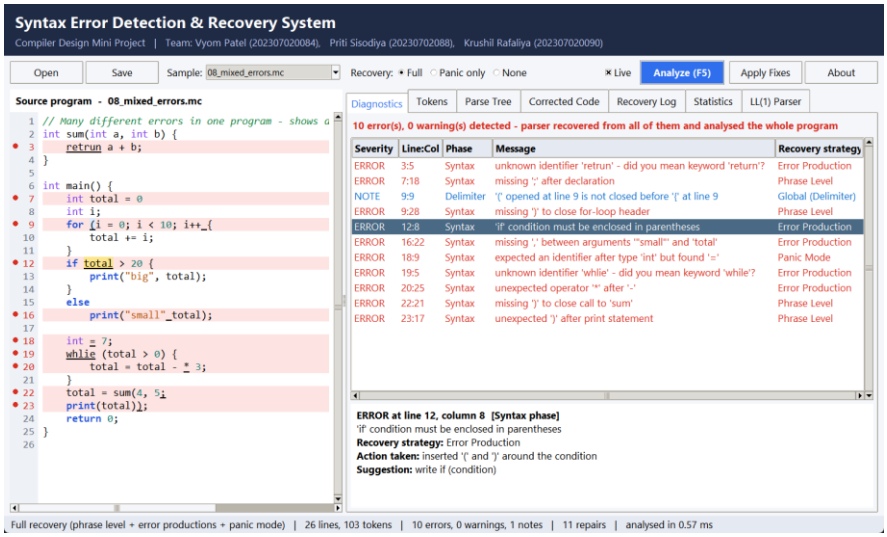


Figure 6.1: Main window with the Diagnostics tab

6.2 Corrected Code

The Corrected Code tab shows the program after every recovery action has been applied. Each change is highlighted in green: inserted semicolons and brackets, retrun corrected to return, whlie to while, and the statement int = 7; discarded by panic mode, which is commented out. *Load into editor* (or *Apply Fixes*) loads this program back into the editor, where it analyses with zero errors.

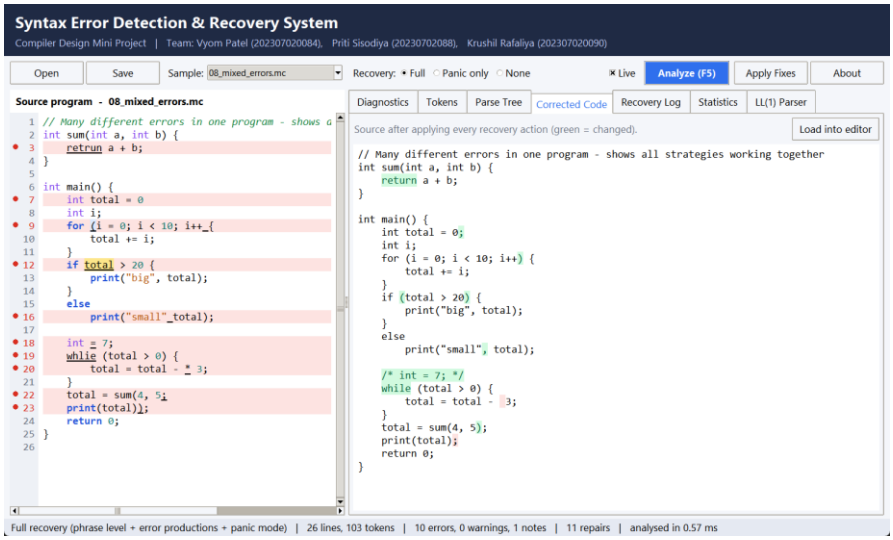


Figure 6.2: Corrected Code tab with repairs highlighted

6.3 Statistics and Comparison of Modes

The Statistics tab shows how many errors were handled by each recovery strategy and compares the three recovery modes on the same program: full recovery reports 10 errors and analyses all 26 lines, panic mode alone reports 11 (one of them a spurious cascading error), and no recovery stops after the first error at line 3.

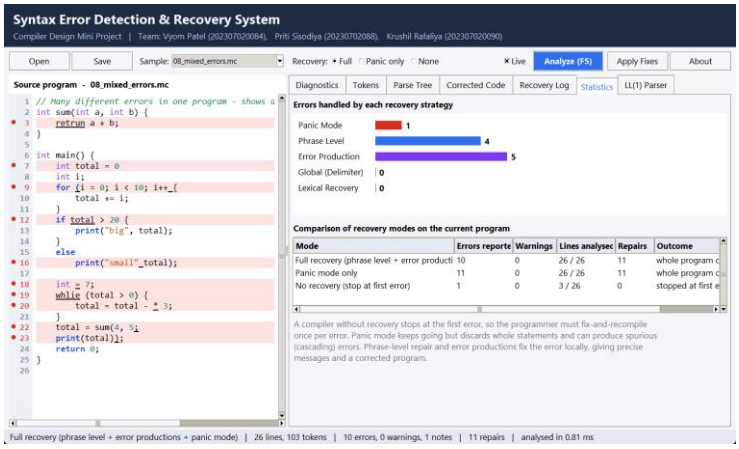


Figure 6.3: Statistics tab: errors per strategy and comparison of modes

6.4 Parse Tree

The Parse Tree tab shows the tree built by the recursive-descent parser for sample 05_condition_errors.mc. Even though the program contains errors, the parser recovers and builds a complete tree; nodes produced by recovery are marked.

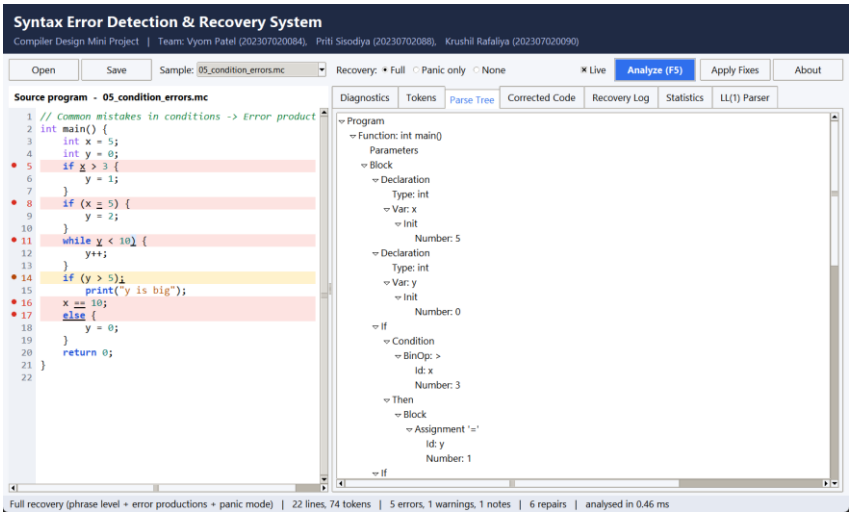


Figure 6.4: Parse Tree tab

6.5 Recovery Log

The Recovery Log shows, step by step, what the parser detected and what it did. Here sample 04_misspelled_keywords.mc is analysed: each misspelled keyword (itn, flaot, whlie, pritn, fi, retrun) is detected and handled by an error production.

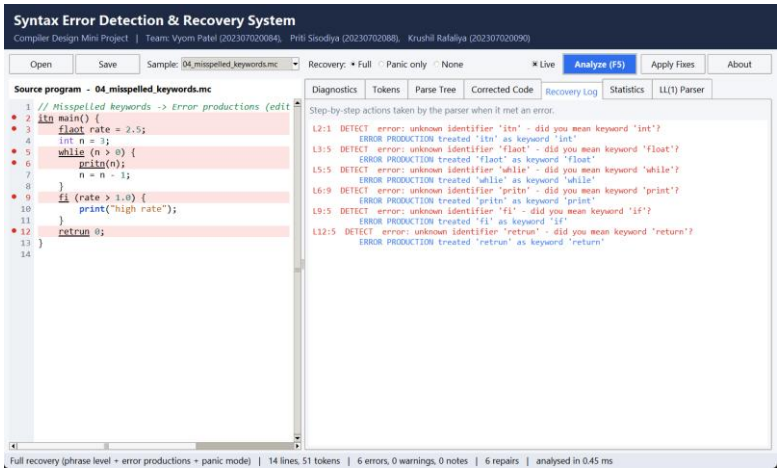


Figure 6.5: Recovery Log tab

6.6 LL(1) Parser: Parsing Trace

The LL(1) Parser tab accepts a grammar and an input string. The trace for a + * b) * 3 shows the stack, the remaining input and the action at every step; the error rows are shown in red. The parser recovers from four errors and accepts the input.

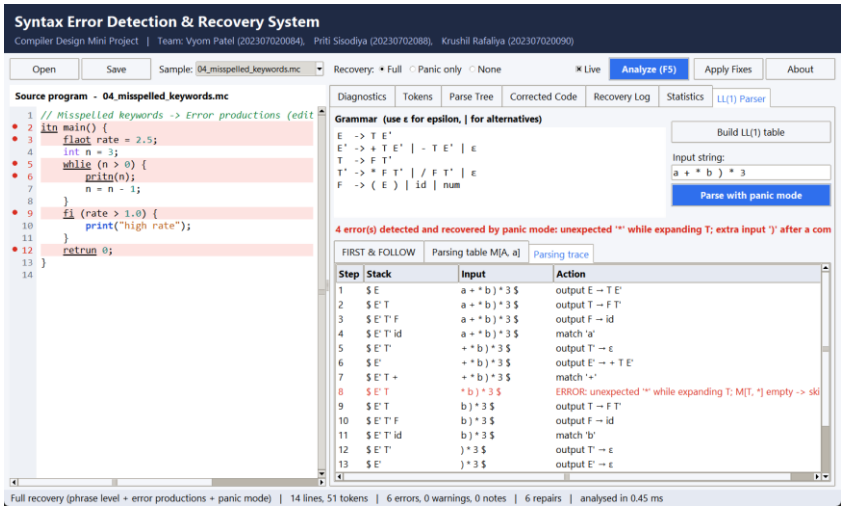


Figure 6.6: LL(1) parsing trace with panic-mode recovery

6.7 LL(1) Parser: Parsing Table

The predictive parsing table M[A, a] built from the FIRST and FOLLOW sets. Empty entries for terminals in FOLLOW(A) are filled with *synch*, which tells the parser where it may safely pop a non-terminal during recovery.

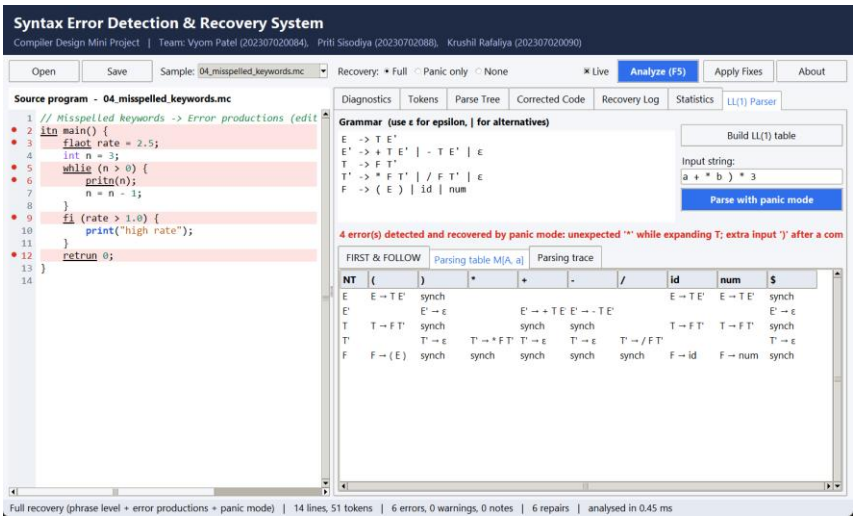


Figure 6.7: LL(1) parsing table with synch entries

6.8 No-Recovery Mode

For comparison, the same program analysed in *No recovery* mode behaves like a naive compiler: it reports only the first error and the rest of the program is not checked.

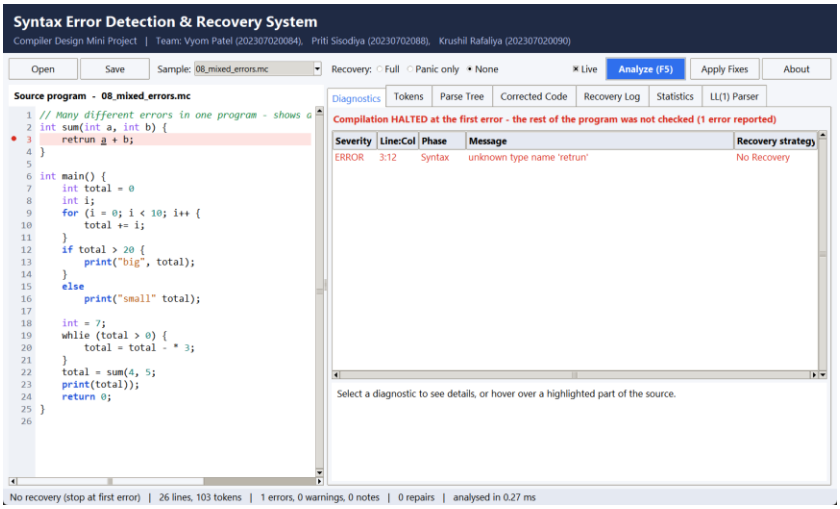


Figure 6.8: No-recovery mode stops at the first error

6.9 Command-Line Interface

The CLI prints the same information as a coloured console report. With --compare it first compares the three modes, then lists each error with the source line, a caret under the error position, the recovery action and a hint.

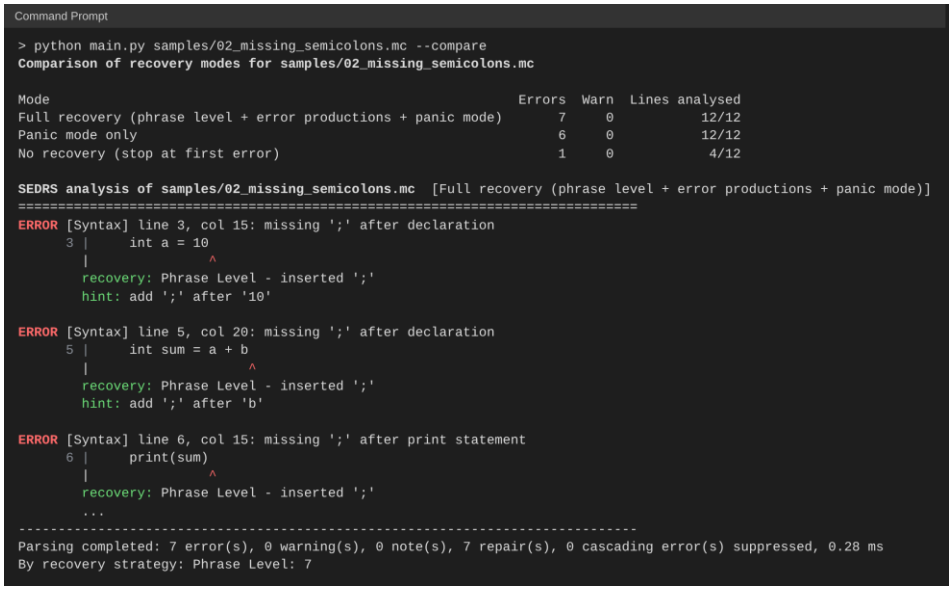


Figure 6.9: Command-line output for sample 02_missing_semicolons.mc

7. RESULTS AND CONCLUSION

7.1 Test Programs

The system was evaluated on eight sample programs shipped in the samples/ folder. Sample 01 is a correct program; each of the others concentrates on one class of error, and sample 08 mixes all of them.

Table 7.1: Sample programs

Sample	Content
01_valid_program	Correct program (factorial, arrays, loops)
02_missing_semicolons	Missing ';', commas used in a for-header
03_unbalanced_delimiters	Unbalanced ( ) [ ] { }
04_misspelled_keywords	itn, flaot, whlie, pritn, fi, retrun
05_condition_errors	Missing parentheses, '=' in a condition, '==' as a statement, stray else
06_expression_errors	Missing operand or operator, double operators
07_lexical_errors	@, $, &, 2ndValue, 3.14.15, 'ab', unterminated string
08_mixed_errors	All kinds of errors in one program

7.2 Results in Full-Recovery Mode

Table 7.2: Results of SEDRS in full-recovery mode

Sample	Errors	Warn.	Handled by	Corrected code
01_valid_program	0	0	-	0 errors
02_missing_semicolons	7	0	PL 7	0 errors
03_unbalanced_delimiters	5	0	PL 5, GD 3 notes	0 errors
04_misspelled_keywords	6	0	EP 6	0 errors
05_condition_errors	5	1	EP 6, GD 1 note	0 errors
06_expression_errors	5	0	EP 2, PM 3	0 errors
07_lexical_errors	10	0	LX 7, PL 2, PM 1, GD 1 note	0 errors
08_mixed_errors	10	0	EP 5, PL 4, PM 1, GD 1 note	0 errors

PL = phrase level, EP = error production, PM = panic mode, LX = lexical recovery, GD = global delimiter analysis. In **every** sample, all errors were detected in a single run, and the generated corrected program re-parses with zero errors.

7.3 Comparison of Recovery Modes

Table 7.3: Errors reported and lines analysed in each mode

Sample	Full recovery	Panic mode only	No recovery
01_valid_program	0 err, 31/31 lines	0 err, 31/31 lines	0 err, 31/31 lines
02_missing_semicolons	7 err, 12/12 lines	6 err, 12/12 lines	1 err, 4/12 lines
03_unbalanced_delimiters	5 err, 14/14 lines	5 err, 14/14 lines	1 err, 4/14 lines
04_misspelled_keywords	6 err, 14/14 lines	5 err, 14/14 lines	1 err, 2/14 lines
05_condition_errors	5 err, 22/22 lines	5 err, 22/22 lines	1 err, 5/22 lines
06_expression_errors	5 err, 14/14 lines	5 err, 14/14 lines	1 err, 5/14 lines
07_lexical_errors	10 err, 14/14 lines	9 err, 14/14 lines	1 err, 4/14 lines
08_mixed_errors	10 err, 26/26 lines	11 err, 26/26 lines	1 err, 3/26 lines

The comparison clearly shows why recovery matters:

- **No recovery** finds only the first error and checks on average less than a third of the program.
- **Panic mode only** checks the whole program, but because it discards complete statements it misses errors inside them (for example 6 instead of 7 in sample 02) and can create **spurious** errors — in sample 08 it reports an extra *'else' without a matching 'if'* because the if-statement it belonged to was discarded.
- **Full recovery** finds every error, gives specific messages and repairs the program so that a corrected version can be produced.

7.4 Example: Before and After Recovery

Sample 02_missing_semicolons.mc is shown below together with the corrected program generated by SEDRS. The seven errors are fixed by phrase-level insertion of ; and by substituting , with ; in the for-header.

Input program:

```
int main() {
    int a = 10
    int b = 20;
    int sum = a + b
    print(sum)
    for (int i = 0, i < 5, i++) {
        sum = sum + i
    }
    return 0
}
```

Corrected program generated by SEDRS:

```
int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;
    print(sum);
    for (int i = 0; i < 5; i++) {
        sum = sum + i;
    }
    return 0;
}
```

7.5 Testing

The project contains **40 automated unit tests** written with Python's unittest framework. They are grouped as shown in Table 7.4. One test re-parses the corrected output of every sample program and checks that it has zero errors; other tests check that ordinary identifiers and function calls (such as foo();) are not wrongly reported as misspelled keywords, and that a statement with a missing body (such as while (x) at the end of the file) is repaired. All 40 tests pass.

Table 7.4: Unit tests

Test group	Tests	What is checked
TestValid	2	Correct programs and every construct give no errors
TestLexical	6	Each lexical error and its recovery
TestPhraseLevel	6	Insertion, substitution and deletion rules
TestErrorProductions	9	Every error production, no false alarms on user functions
TestPanicMode	3	Statement discarding and continued parsing

Test group	Tests	What is checked
TestModesAndGlobal	3	The three modes and delimiter root-cause notes
TestCorrectedCode	1	Corrected code of every sample has 0 errors
TestBugFixes	5	No false keyword alarm on calls like foo(); missing body repaired
TestLL1	5	FIRST/FOLLOW values, synch entries, recovery, conflict detection
Total	40	All tests pass

7.6 Performance

Analysis is very fast: each sample program, including lexing, delimiter analysis, parsing with recovery, applying repairs and re-parsing the corrected program, takes about **1 millisecond**. This satisfies the third goal of an error handler — minimal overhead — and makes the *Live* mode of the GUI possible, where the program is re-analysed on every keystroke.

7.7 Limitations

- MiniC is a teaching subset of C; it has no structures, pointers, multi-dimensional arrays or pre-processor.
- Semantic errors (types, undeclared variables) are not checked.
- Error productions only cover mistakes that were anticipated; an unusual error falls back to panic mode, which may discard a statement.
- Global correction is used in a restricted form (delimiter balancing) rather than as a full minimum-distance correction, which would be too expensive.

7.8 Conclusion

The Syntax Error Detection and Recovery System meets all the objectives of the project. It detects lexical and syntax errors with precise positions and clear, helpful messages; it recovers from every error so that all errors in a program are reported in one run; it suppresses cascading errors and always terminates; and it produces a corrected program that compiles cleanly.

The results show that **combining recovery strategies gives far better results than any single one**. Phrase-level repair and error productions handle common, local mistakes precisely and make automatic correction possible; panic mode is the safety net that guarantees progress when nothing else applies; and the global delimiter pass explains the root cause of bracket errors that a left-to-right parser discovers too late. The table-driven LL(1) demonstrator connects the implementation to the theory of FIRST and FOLLOW sets and synch entries studied in the course. Through its GUI and CLI, the project demonstrates the practical working of error handling in the syntax analysis phase of a compiler.

7.9 Future Scope

- Add a **semantic analysis** phase with a symbol table to detect undeclared variables and type mismatches.
- Extend MiniC with structures, pointers and multi-dimensional arrays.
- Add an **LR (SLR/LALR) parser** demonstrator with its own error-recovery routines, to compare bottom-up and top-down recovery.
- Rank several candidate repairs and choose the cheapest one, moving closer to true global correction.
- Generate intermediate (three-address) code for programs that are free of errors.
- Provide the system as an editor plug-in or a web application.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson Education, 2007 (Chapter 4: Syntax Analysis — error recovery strategies, FIRST and FOLLOW, predictive parsing).

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed., Morgan Kaufmann, 2011.

[3] D. Grune and C. J. H. Jacobs, *Parsing Techniques: A Practical Guide*, 2nd ed., Springer, 2008.

[4] F. J. Damerau, "A technique for computer detection and correction of spelling errors," *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964.

[5] Python Software Foundation, *Python 3 Documentation — tkinter, unittest, dataclasses, argparse*, https://docs.python.org/3/.